



Algorithmes de tri

Support de lecture + TP fil rouge

Développement Web · 2025–2026 | À lire avant la séance · TP à rendre

Comment utiliser ce document :

- **Parties 1 et 2** (pages suivantes) : à lire seul avant la séance. Lisez attentivement, comprenez les traces d'exécution, recopiez les fonctions dans `tris.php` et vérifiez que vos tests passent.
- **Partie 3** : exercices à réaliser en séance, sur machine.

Fil rouge : Tout au long de ce TP, vous allez appliquer les algorithmes de tri à un cas concret : **la gestion des notes des étudiants**. Les données et fonctions utilitaires viennent directement du TP Fonctions PHP — vous allez les réutiliser et les enrichir.

Prérequis : TP Fonctions PHP (bibliothèque `notes.php`) complète.

Rendu : Dossier `tp-tris/` dans votre dépôt Git. Commit : `tp-tris : bibliothèque + fil rouge`.

Partie 1 — Vocabulaire

Avant d'étudier les algorithmes, voici les termes que nous utiliserons tout au long de ce document. Chaque terme a une définition précise — apprenez-les, vous en aurez besoin pour comparer les tris.

Tri sur place

Un tri est dit **sur place** (*in place*) s'il n'utilise pas de tableau supplémentaire pour faire son travail. Il réorganise les éléments directement dans le tableau d'origine.

Analogie : trier des cartes dans sa main sans poser de pile à côté.

Tri stable

Un tri est dit **stable** si, lorsque deux éléments ont la même valeur, leur ordre relatif dans le tableau est préservé après le tri.

Exemple : si Amina (10/20) apparaît avant Mehdi (10/20) dans la liste, un tri stable garantit qu'Amina reste avant Mehdi après le tri.

Cas favorable / Cas défavorable

- **Cas favorable** : la configuration du tableau d'entrée qui minimise le nombre d'opérations. Souvent : le tableau est déjà trié.
- **Cas défavorable** : la configuration qui maximise le nombre d'opérations. Souvent : le tableau est trié à l'envers.

Ordre de grandeur

Plutôt que de compter exactement le nombre d'opérations, on donne un **ordre de grandeur** en fonction de la taille n du tableau.

L'opération que l'on compte est la **comparaison entre deux éléments du tableau** — du type `$tab[$j] > $tab[$j+1]` — car c'est elle qui détermine si un échange est nécessaire, et c'est ce que vous observez directement dans les traces d'exécution.

Cet ordre de grandeur s'écrit avec la **notation grand O** : $O(f(n))$. Elle se lit “de l'ordre de $f(n)$ ” et exprime comment le nombre de comparaisons **évolue** quand n grandit — pas sa valeur exacte.

*Exemple : un tri en $O(n^2)$ effectue environ $n \times n$ comparaisons. Il en fera environ 100 pour $n = 10$, environ 10 000 pour $n = 100$, et environ 1 000 000 pour $n = 1000$. On dit que cet algorithme est **quadratique**.*

À l'opposé, un tri en $O(n \log n)$ en effectuera environ 33 pour $n = 10$, environ 700 pour $n = 100$, et environ 10 000 pour $n = 1000$. On dit que cet algorithme est **quasi-linéaire** — c'est beaucoup plus raisonnable.

Diviser pour mieux régner

Principe algorithmique qui consiste à **découper un problème difficile en sous-problèmes plus simples**, résoudre chaque sous-problème indépendamment, puis **combinaison** les résultats.

Analogie : pour trier une classe de 30 étudiants, on divise en deux groupes de 15, on trie chaque groupe séparément, puis on fusionne les deux groupes triés.

Les tris qui utilisent ce principe (tri fusion, tri rapide) sont généralement beaucoup plus rapides que les tris naïfs sur de grands tableaux.

Partie 2 — Les algorithmes

Nous étudions cinq algorithmes de tri. Les trois premiers sont des **tris naïfs** — simples à comprendre et à coder, mais lents sur de grands tableaux. Les deux derniers sont des **tris diviser pour mieux régner** — plus complexes, mais bien plus efficaces.

Tri à bulles

Caractéristiques

Famille	Tri naïf
Sur place	Oui
Stable	Oui

Principe

On parcourt le tableau de gauche à droite en comparant chaque élément avec son voisin de droite. Si les deux sont dans le mauvais ordre, on les échange. On répète ce parcours jusqu'à ce que le tableau soit entièrement trié. À chaque passage, le plus grand élément “remonte” vers la fin — comme une bulle qui monte à la surface de l'eau.

Trace d'exécution sur [5, 3, 8, 1, 4]

Tableau initial : [5, 3, 8, 1, 4]

Passage 1 :

[5, 3, 8, 1, 4] → échange 5 et 3 → [3, 5, 8, 1, 4]
[3, 5, 8, 1, 4] → $5 < 8$, pas d'échange
[3, 5, 8, 1, 4] → échange 8 et 1 → [3, 5, 1, 8, 4]
[3, 5, 1, 8, 4] → échange 8 et 4 → [3, 5, 1, 4, 8] ← 8 est en place

Passage 2 :

[3, 5, 1, 4, 8] → $3 < 5$, pas d'échange
[3, 5, 1, 4, 8] → échange 5 et 1 → [3, 1, 5, 4, 8]
[3, 1, 5, 4, 8] → échange 5 et 4 → [3, 1, 4, 5, 8] ← 5 est en place

Passage 3 :

[3, 1, 4, 5, 8] → échange 3 et 1 → [1, 3, 4, 5, 8]
[1, 3, 4, 5, 8] → $3 < 4$, pas d'échange ← 4 est en place

Passage 4 :

[1, 3, 4, 5, 8] → $1 < 3$, pas d'échange ← tableau trié

Résultat final : [1, 3, 4, 5, 8] ✓

Implémentation PHP — à recopier dans tris.php

Listing 1 – Tri à bulles

```
1 function triBulles($tab) {  
2     $n = count($tab);  
3     for ($i = 0; $i < $n - 1; $i++) {  
4         for ($j = 0; $j < $n - 1 - $i; $j++) {  
5             if ($tab[$j] > $tab[$j + 1]) {
```

```
6         $temp      = $tab[$j];
7         $tab[$j]    = $tab[$j + 1];
8         $tab[$j + 1] = $temp;
9     }
10 }
11 }
12 return $tab;
13 }
```

Test à exécuter

Listing 2 – Test tri à bulles

```
1 $tab = [5, 3, 8, 1, 4];
2 $trie = triBulles($tab);
3 echo implode(', ', $trie); // attendu : 1, 3, 4, 5, 8
```

Étude expérimentale de la complexité

Apprendre à **analyser la complexité** d'un algorithme est essentiel : c'est ce qui permet de **comparer plusieurs algorithmes** entre eux et de choisir le bon pour un problème donné.

Nous le faisons ici en détail uniquement pour le tri à bulles — la démarche serait identique pour les autres tris.

Étape 1 — Compter les comparaisons. Le code suivant est une version instrumentée de `triBulles()` qui **compte** le nombre de comparaisons. La ligne qui incrémente `$compteur` est manquante : à vous de la placer au bon endroit.

Listing 3 – Tri à bulles instrumenté — à compléter

```
1 function triBullesCompte($tab) {
2     $n      = count($tab);
3     $compteur = 0;
4     for ($i = 0; $i < $n - 1; $i++) {
5         for ($j = 0; $j < $n - 1 - $i; $j++) {
6             // ??? incrementer $compteur ici ???
7             if ($tab[$j] > $tab[$j + 1]) {
8                 $temp      = $tab[$j];
9                 $tab[$j]    = $tab[$j + 1];
10                $tab[$j + 1] = $temp;
11            }
12        }
13    }
14    return $compteur;
15 }
```

Étape 2 — Mesurer le temps d'exécution. Cette deuxième fonction exécute le vrai `triBulles()` et mesure son **temps d'exécution** en millisecondes :

Listing 4 – Chronométrage du tri à bulles

```
1 function triBullesChrono($tab) {
2     $debut = microtime(true);
3     triBulles($tab);
4     $fin    = microtime(true);
5     return round(($fin - $debut) * 1000, 2); // en ms
6 }
```

Étape 3 — Lancer les mesures. Appelez les deux fonctions sur des tableaux triés à l'envers (cas défavorable) de tailles croissantes :

Listing 5 – Lancement des mesures

```
1 $tailles = [100, 500, 1000, 2000, 5000, 10000];
2
3 foreach ($tailles as $n) {
4     $tab      = range($n, 1); // [n, n-1, ..., 2, 1] = cas
        defavorable
5     $nbComp = triBullesCompte($tab);
6     $temps  = triBullesChrono($tab);
7     echo "n = $n : $nbComp comparaisons, $temps ms\n";
8 }
```

Attention

Pour les grandes tailles (5 000 et 10 000), l'exécution peut prendre **plusieurs secondes** — c'est normal et c'est même le but : vous allez **ressentir** la lenteur du tri à bulles.

Étape 4 — Analyser les résultats. Complétez le tableau ci-dessous avec vos mesures :

n	Comparaisons	$n \times n$	Temps (ms)
100		10 000	
500		250 000	
1 000		1 000 000	
2 000		4 000 000	
5 000		25 000 000	
10 000		100 000 000	

Questions :

1. Que constatez-vous en comparant la colonne “Comparaisons” et la colonne “ $n \times n$ ” ?

2. Quand n est multiplié par 10 (de 100 à 1 000, puis de 1 000 à 10 000), par combien le nombre de comparaisons est-il multiplié ? Et le temps d'exécution ?
3. Y a-t-il une **corrélation visible** entre le nombre de comparaisons et le temps d'exécution ? Pourquoi est-ce attendu ?
4. En déduire la notation grand O du tri à bulles dans le cas défavorable.

Pour les autres tris. Dans le tableau récapitulatif (Partie 3), nous indiquerons directement l'ordre de grandeur de chaque tri sans refaire l'analyse complète. Mais sachez que la **démarche est exactement la même** : on identifie l'opération à compter (la comparaison), on l'instrumente, on mesure, on compare à des fonctions connues (n , n^2 , $n \log n$), on conclut.

Ce que vous venez de faire s'appelle l'étude de la **complexité** d'un algorithme. Plus précisément, la complexité s'intéresse au **comportement asymptotique** : comment l'algorithme se comporte quand n devient **très grand** ? C'est pour ça qu'on ne compte pas exactement — ce qui compte, c'est la tendance quand n grandit, pas la valeur exacte pour $n = 5$.

Tri par sélection

Caractéristiques

Famille	Tri naïf
Sur place	Oui
Stable	Non

Principe

On cherche le minimum dans tout le tableau et on le place en première position. Puis on cherche le minimum dans le reste du tableau (sans la première case) et on le place en deuxième position. On répète jusqu'à ce que le tableau soit trié. À chaque étape, on **sélectionne** le plus petit élément restant.

Trace d'exécution sur [5, 3, 8, 1, 4]

Tableau initial : [5, 3, 8, 1, 4]

Étape 1 : minimum de [5,3,8,1,4] = 1 (position 3) → échange avec position 0
[1, 3, 8, 5, 4]

Étape 2 : minimum de [3,8,5,4] = 3 (position 1) → déjà en place
[1, 3, 8, 5, 4]

Étape 3 : minimum de [8,5,4] = 4 (position 4) → échange avec position 2
[1, 3, 4, 5, 8]

Étape 4 : minimum de $[5, 8] = 5 \rightarrow$ déjà en place

$[1, 3, 4, 5, 8]$ tableau trié

Résultat final : $[1, 3, 4, 5, 8]$ ✓

Implémentation PHP — à recopier dans `tris.php`

Listing 6 – Tri par sélection

```
1 function triSelection($tab) {  
2     $n = count($tab);  
3     for ($i = 0; $i < $n - 1; $i++) {  
4         $indiceMin = $i;  
5         for ($j = $i + 1; $j < $n; $j++) {  
6             if ($tab[$j] < $tab[$indiceMin]) {  
7                 $indiceMin = $j;  
8             }  
9         }  
10        if ($indiceMin !== $i) {  
11            $temp = $tab[$i];  
12            $tab[$i] = $tab[$indiceMin];  
13            $tab[$indiceMin] = $temp;  
14        }  
15    }  
16    return $tab;  
17 }
```

Test à exécuter

Listing 7 – Test tri par sélection

```
1 $tab = [5, 3, 8, 1, 4];  
2 $trie = triSelection($tab);  
3 echo implode(', ', $trie); // attendu : 1, 3, 4, 5, 8
```

Tri par insertion

Caractéristiques

Famille	Tri naïf
Sur place	Oui
Stable	Oui

Principe

On construit le tableau trié élément par élément, de gauche à droite. Chaque nouvel élément est **inséré** à sa juste place dans la partie gauche, déjà triée. C'est exactement ce qu'on fait intuitivement quand on trie des cartes dans sa main : on prend une nouvelle carte et on la glisse au bon endroit parmi celles déjà triées.

Trace d'exécution sur [5, 3, 8, 1, 4]

Tableau initial : [5, 3, 8, 1, 4]

Étape 1 : partie triée = [5] | insérer 3 → $3 < 5$, on décale 5

Partie triée : [3, 5]

Étape 2 : partie triée = [3, 5] | insérer 8 → $8 > 5$, on place 8 après

Partie triée : [3, 5, 8]

Étape 3 : partie triée = [3, 5, 8] | insérer 1 → $1 < 8, < 5, < 3$, on place 1 au début

Partie triée : [1, 3, 5, 8]

Étape 4 : partie triée = [1, 3, 5, 8] | insérer 4 → $4 < 5, > 3$, on place 4 entre 3 et 5

Partie triée : [1, 3, 4, 5, 8]

Résultat final : [1, 3, 4, 5, 8] ✓

Implémentation PHP — à recopier dans tris.php

Listing 8 – Tri par insertion

```
1 function triInsertion($tab) {
2     $n = count($tab);
3     for ($i = 1; $i < $n; $i++) {
4         $cle = $tab[$i];
5         $j = $i - 1;
6         while ($j >= 0 && $tab[$j] > $cle) {
7             $tab[$j + 1] = $tab[$j];
8             $j--;
9         }
10        $tab[$j + 1] = $cle;
11    }
12    return $tab;
13 }
```

Test à exécuter

Listing 9 – Test tri par insertion

```
1 $tab = [5, 3, 8, 1, 4];
```



```

2 $trie = triInsertion($tab);
3 echo implode(', ', $trie); // attendu : 1, 3, 4, 5, 8

```

Tri fusion

Caractéristiques

Famille	Diviser pour mieux régner
Sur place	Non (utilise des tableaux intermédiaires)
Stable	Oui

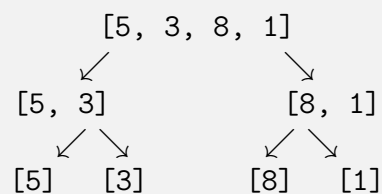
Principe

On **divise** le tableau en deux moitiés. On trie chaque moitié indépendamment (en rappelant le même algorithme récursivement). Puis on **fusionne** les deux moitiés triées en un seul tableau trié. La fusion de deux tableaux déjà triés est une opération simple et rapide : on compare les premiers éléments des deux tableaux et on prend toujours le plus petit.

Schéma — diviser puis fusionner

Tableau initial : [5, 3, 8, 1]

Phase 1 — Diviser :



Phase 2 — Fusionner :

$[5] + [3] \rightarrow [3, 5]$
 $[8] + [1] \rightarrow [1, 8]$
 $[3, 5] + [1, 8] \rightarrow [1, 3, 5, 8] \checkmark$

Implémentation PHP — à recopier dans tris.php

Listing 10 – Tri fusion

```

1 function triFusion($tab) {

```

```
2  $n = count($tab);
3  if ($n <= 1) return $tab;
4
5  $milieu = (int)($n / 2);
6  $gauche = triFusion(array_slice($tab, 0, $milieu));
7  $droite = triFusion(array_slice($tab, $milieu));
8
9  return fusionner($gauche, $droite);
10 }
11
12 function fusionner($gauche, $droite) {
13     $resultat = [];
14     $i = 0;
15     $j = 0;
16
17     while ($i < count($gauche) && $j < count($droite)) {
18         if ($gauche[$i] <= $droite[$j]) {
19             $resultat[] = $gauche[$i++];
20         } else {
21             $resultat[] = $droite[$j++];
22         }
23     }
24
25     while ($i < count($gauche)) $resultat[] = $gauche[$i++];
26     while ($j < count($droite)) $resultat[] = $droite[$j++];
27
28     return $resultat;
29 }
```

Test à exécuter

Listing 11 – Test tri fusion

```
1 $tab = [5, 3, 8, 1, 4];
2 $trie = triFusion($tab);
3 echo implode(', ', $trie); // attendu : 1, 3, 4, 5, 8
```

Tri rapide

Caractéristiques

Famille	Diviser pour mieux régner
Sur place	Non (dans cette implémentation)
Stable	Non

Principe

On choisit un élément du tableau appelé **pivot**. On place tous les éléments plus petits que le pivot à sa gauche, et tous les plus grands à sa droite. Le pivot est alors à sa position définitive. On applique ensuite le même procédé récursivement sur la partie gauche et sur la partie droite.

Schéma — pivot = dernier élément

Tableau initial : [5, 3, 8, 1, 4] — pivot = 4

Étape 1 : on répartit autour du pivot 4

[3, 1] [4] [5, 8]
(éléments < 4) (pivot) (éléments > 4)

Étape 2 : on trie récursivement chaque partie

tri([3,1]) → [1,3]
tri([5,8]) → [5,8]

Étape 3 : on assemble

[1, 3] + [4] + [5, 8] = [1, 3, 4, 5, 8] ✓

Implémentation PHP — à recopier dans tris.php

Listing 12 – Tri rapide

```
1 function triRapide($tab) {  
2     $n = count($tab);  
3     if ($n <= 1) return $tab;  
4  
5     $pivot = $tab[$n - 1];  
6     $gauche = [];  
7     $droite = [];  
8  
9     for ($i = 0; $i < $n - 1; $i++) {  
10         if ($tab[$i] <= $pivot) {  
11             $gauche[] = $tab[$i];  
12         } else {  
13             $droite[] = $tab[$i];  
14         }  
15     }  
16 }
```

```

17     return array_merge(triRapide($gauche), [$pivot], triRapide(
18         $droite));
    }

```

Test à exécuter

Listing 13 – Test tri rapide

```

1 $tab = [5, 3, 8, 1, 4];
2 $trie = triRapide($tab);
3 echo implode(', ', $trie); // attendu : 1, 3, 4, 5, 8

```

Partie 3 — Exercices en séance

Les exercices suivants sont à réaliser sur machine, en séance. Votre fichier `tris.php` doit déjà contenir les 5 fonctions recopiées depuis la Partie 2, et tous vos tests doivent passer avant de commencer.

Exercice 1. Tableau comparatif des tris

Sans exécuter de code, complétez le tableau suivant en vous basant sur ce que vous avez lu dans la Partie 2. Répondez dans un fichier `comparaison.txt`.

Algorithme	Famille	Sur place	Stable	Cas défavorable
Tri à bulles				
Tri par sélection				
Tri par insertion				
Tri fusion				
Tri rapide				

Exercice 2. Chronométrage et vérification des ordres de grandeur

Vous allez mesurer les temps d'exécution de chaque tri sur des tableaux de tailles croissantes, et vérifier que les ordres de grandeur donnés précédemment correspondent bien à la réalité.

Code de départ

Créez un fichier `chrono.php` et recopiez ce code :

Listing 14 – Squelette du chronométrage

```

1 <?php
2 require_once 'tris.php';
3
4 $tailles = [100, 500, 1000, 3000];
5 $tris    = ['triBulles', 'triSelection', 'triInsertion', '
6           triFusion', 'triRapide'];
7
8 foreach ($tailles as $n) {
9     $tableau = [];
10    for ($i = 0; $i < $n; $i++) {
11        $tableau[] = rand(1, 100000);
12    }
13
14    echo "--- Taille : $n ---\n";
15
16    foreach ($tris as $tri) {
17        $copie = $tableau;
18        $debut = microtime(true);
19        $tri($copie);
20        $fin   = microtime(true);
21        $duree = round(($fin - $debut) * 1000, 2);
22        echo "$tri : $duree ms\n";
23    }
24    echo "\n";
25 }
?>

```

Ce que vous devez faire

1. Exécuter `chrono.php` en ligne de commande : `php chrono.php`
2. Relever les temps obtenus et remplir le tableau suivant dans `comparaison.txt` :

Tri	n=100	n=500	n=1000	n=3000
Tri à bulles				
Tri par sélection				
Tri par insertion				
Tri fusion				
Tri rapide				

Question à répondre dans `comparaison.txt` : En observant vos résultats, que constatez-vous quand la taille est multipliée par 10 pour les tris naïfs ? Et pour les tris diviser pour mieux régner ? Est-ce cohérent avec les ordres de grandeur donnés en Partie 2 ?

Exercice 3. Fil rouge — Trier la liste des étudiants

Vous allez intégrer votre bibliothèque de tris dans le projet “Gestion des notes”. Repartez du fichier `index.php` du TP Fonctions. Le tableau `$etudiants` est déjà disponible — pas de CSV pour l’instant, on travaille toujours avec les données codées en dur.

Ce que vous devez produire

Créez un fichier `classement.php` qui affiche trois classements distincts de la promotion, chacun obtenu avec un algorithme de tri différent.

Classement 1 — Par moyenne décroissante (tri fusion)

Triez les étudiants du meilleur au moins bon. Utilisez **obligatoirement** `triFusion()`. Sortie attendue :

```
=== Classement par moyenne (meilleur au moins bon) ===
1. Mehdi TAZI          --- 17.00/20 --- Très Bien   --- ADMIS
2. Omar SEKKAT         --- 14.00/20 --- Bien       --- ADMIS
3. Yassine ELBOUARI    --- 12.20/20 --- Assez Bien --- ADMIS
4. Hamza BENKIRAN      --- 10.80/20 --- Passable  --- ADMIS
5. Nour IDRISI         --- 10.00/20 --- Passable  --- ADMIS
6. Salma CHRAIBI      --- 7.00/20  --- Insuffisant --- AJOURNÉ
7. Lina FASSI          --- 5.00/20  --- Insuffisant --- AJOURNÉ
```

Classement 2 — Par nom alphabétique (tri par insertion)

Triez les étudiants par ordre alphabétique du nom. Utilisez **obligatoirement** `triInsertion()`.

Classement 3 — Par note dans une matière (tri rapide)

Demandez à l'utilisateur de saisir un numéro de matière (0 = Maths, 1 = Physique, etc.) via `$_GET` ou `readline()`, puis triez les étudiants par note dans cette matière, du plus élevé au plus bas. Utilisez **obligatoirement** `triRapide()`.

Contraintes

- Chaque classement utilise l’algorithme imposé — pas de `sort()` PHP natif
- Vous devez écrire des **fonctions de comparaison** pour adapter les tris (qui ne trient que des entiers) à des tableaux associatifs d’étudiants — voir le coup de pouce ci-dessous
- Les affichages réutilisent les fonctions de `notes.php` (`calculerMoyenne()`, `formaterNomComplet()` etc.)

Coup de pouce

Vos fonctions de tri travaillent sur des tableaux de nombres. Pour trier des étudiants, vous devez d’abord extraire les valeurs de tri dans un tableau de nombres, trier ce tableau, puis reconstruire l’ordre des étudiants.

Exemple pour extraire les moyennes :

```
1 // Construire un tableau de moyennes dans le même ordre que
   $etudiants
2 $moyennes = [];
3 foreach ($etudiants as $etudiant) {
4     $moyennes[] = calculerMoyenne($etudiant['notes']);
5 }
6
7 // Trier les moyennes avec triFusion
8 $moyennesTrie = triFusion($moyennes);
9
10 // Retrouver l'ordre des étudiants correspondant
11 // (à vous de construire cette logique)
```

Attention

La reconstruction de l’ordre des étudiants à partir du tableau trié des moyennes est la partie la plus délicate de cet exercice. Réfléchissez bien avant de coder. Si deux étudiants ont la même moyenne, votre code doit gérer ce cas sans planter.